

Part 1: Installing the Chrome Extension (Developer Mode)

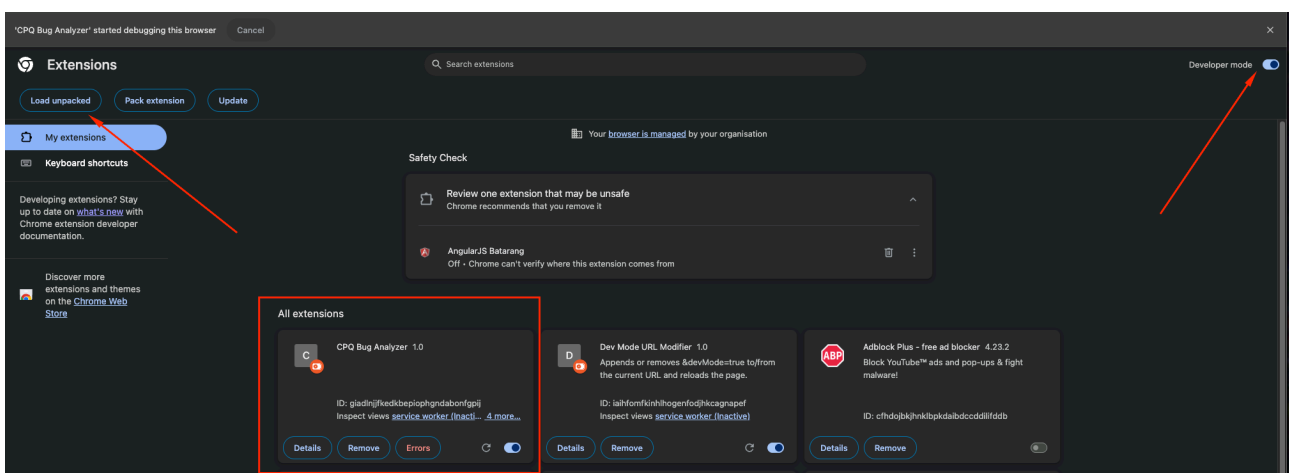
To install your extension, you will need to enable Developer Mode in your Chrome browser and load the unpacked extension.

Step 1: Open the Extensions Page and Enable Developer Mode

1. Open your Chrome browser.
2. Navigate to `chrome://extensions` in your address bar, or access it via the Chrome menu (three vertical dots) -> More Tools -> Extensions.
3. On the Extensions page, locate the "Developer mode" toggle switch in the top-right corner and enable it.

Step 2: Load the Unpacked Extension

1. Once Developer Mode is enabled, a "Load unpacked" button will appear on the top left of the Extensions page. Click this button.
2. A file dialog will open. Select the folder containing your extension's files (this folder should contain the `manifest.json` file).
3. After selecting the folder, your extension will appear as a new card on the Extensions page, similar to the "CPQ Bug Analyzer" card shown in the screenshot.



Step 4: Pin the Extension (Optional but Recommended)

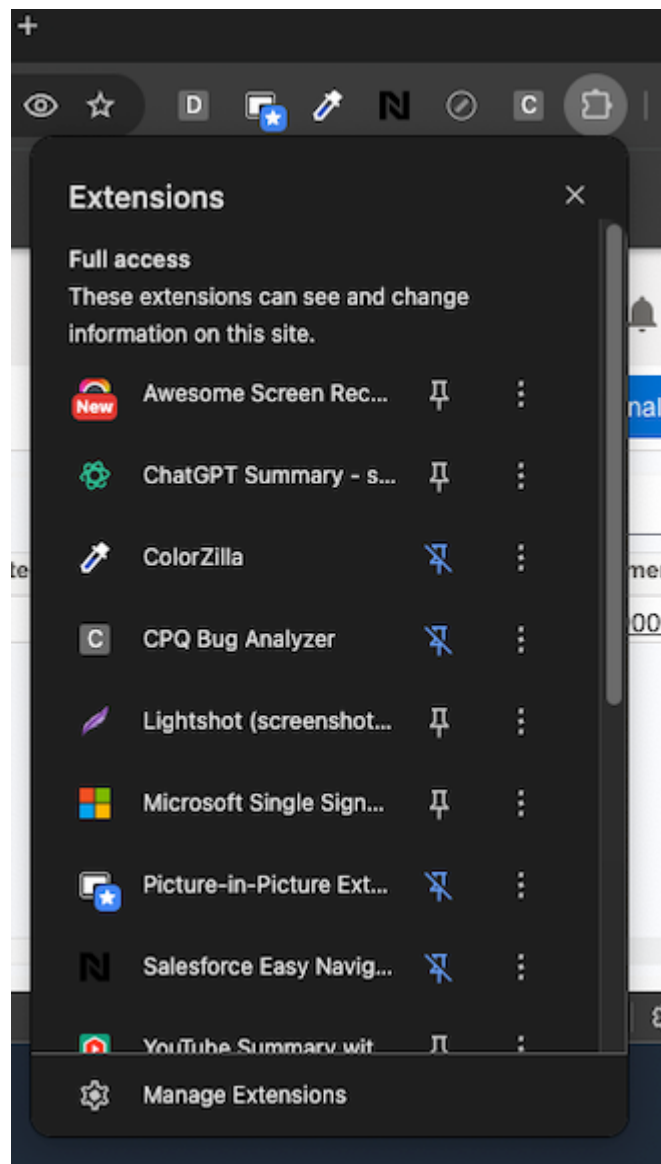
By default, newly installed extensions might be hidden in the Extensions menu (the puzzle piece icon). To easily access your extension, you can pin it to the Chrome toolbar:

1. Click the puzzle piece icon (Extensions menu) in your Chrome toolbar.
2. Find your extension in the list.
3. Click the pin icon next to your extension's name. This will make your extension's icon visible on the toolbar.

Step 5: Enable/Disable the Extension from the Browser Toolbar

Once pinned, you can quickly enable or disable your extension directly from the Chrome toolbar:

1. Click the extension's icon on the toolbar.
2. A dropdown menu will appear, showing a list of your installed extensions. You can see the status of each extension (enabled/disabled) and toggle it on or off.

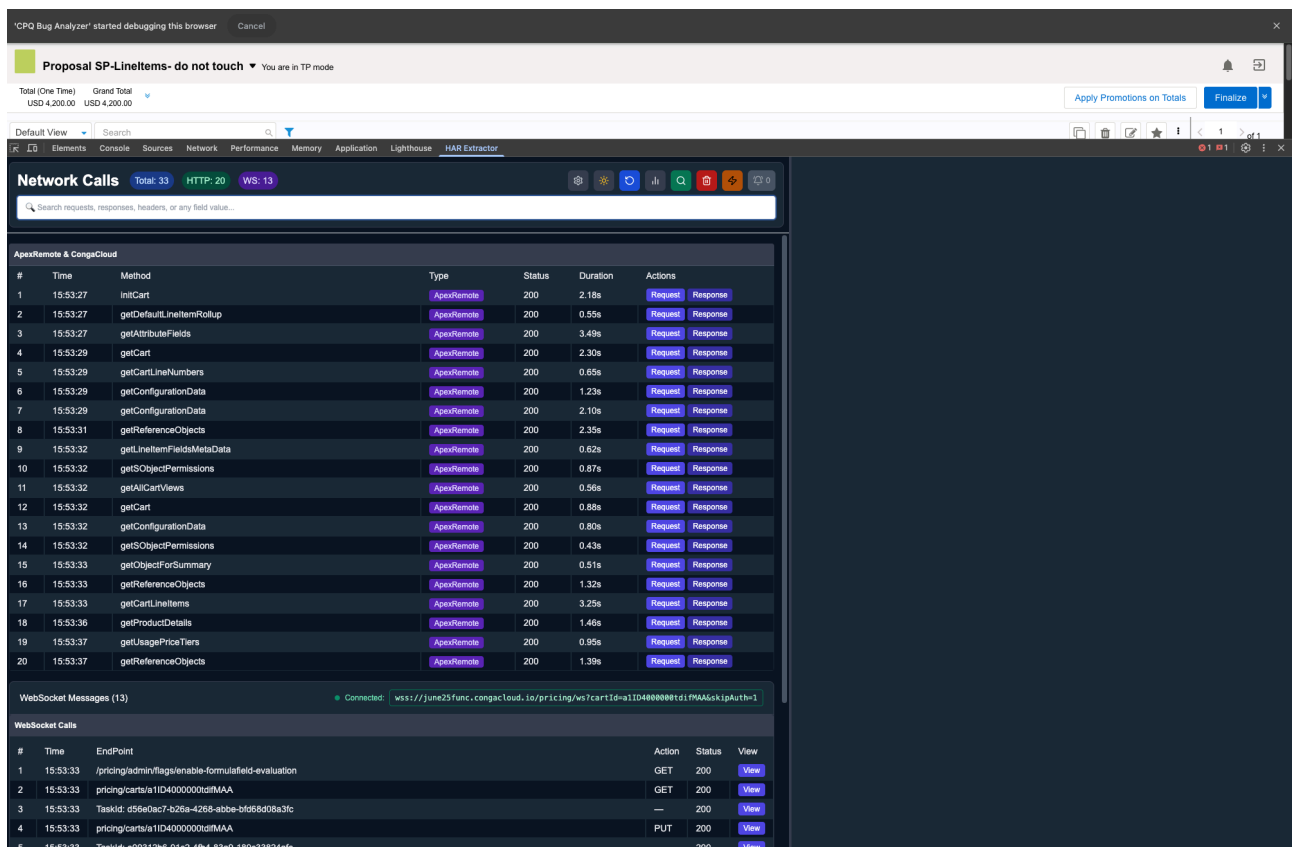


Part 2: Using Your Extension

This section describes how to use the "CPQ Bug Analyzer" extension, specifically focusing on its Network tool tab.

Network Tool Tab Overview

Upon installing and enabling the extension, a new tab named "HAR Extractor" (or similar, depending on the extension's specific name for this feature) will be created in your browser's developer tools. This tab is designed to read and display network calls, including WebSocket (WS) calls.



Important: Debugging Notification

As soon as you open this tab, a notification will appear, typically stating something like "CPQ Bug Analyzer started debugging this browser." **It is crucial not to close this notification.** If you close this notification, the extension will stop reading WebSocket calls, and they will not be displayed in the Network tool tab.

This tab acts as a central point for monitoring the network activity related to your extension, which is essential for debugging and understanding its behavior.

Network Calls Tab Features

The Network Calls tab provides several tools to help you analyze the network traffic:

- **Search Bar:** Located at the top, this bar allows you to "Search with anything to filter the calls" (e.g., search requests, responses, headers, or any field value).
- **HTTP Calls Section:** This main section displays a list of HTTP calls made by the browser, including details like Time, Method, Type, Status, Duration, and Actions (Request/Response).

- **WebSocket Messages Section:** Below the HTTP calls, this section lists "WebSocket calls list," showing WebSocket messages with their Time, Endpoint, Action, Status, and a View option.

In the top right corner of the Network Calls tab, you'll find several buttons:

- **Add Rules:** This button allows you to "Add Rules" for filtering or managing network calls.
- **Search entire Network calls with conditions:** This button enables a more advanced search functionality, allowing you to search the entire network calls with specific conditions.
- **Check history of a particular field:** Use this button to view the history of a particular field within the network calls.
- **Track the network calls with URL patterns:** This feature helps you track network calls based on defined URL patterns.

The screenshot shows the 'Network Calls' tab in a browser's developer tools. The main table lists HTTP calls with columns for #, Time, Method, Type, Status, Duration, and Actions. Below this is a 'WebSocket Messages' section and a 'WebSocket Calls' section. Red arrows point to specific UI elements: a search bar at the top, 'Add Rules', 'Search entire Network calls with conditions', 'Check history of an particular field', and 'Track the network calls with URL patterns'.

#	Time	Method	Type	Status	Duration	Actions
1	15:53:27	InitCart	ApexRemote	200	2.18s	Request Response
2	15:53:27	getDefaultLineItemRollup	ApexRemote	200	0.55s	Request Response
3	15:53:27	getAttributeFields	ApexRemote	200	3.49s	Request Response
4	15:53:29	getCart	ApexRemote	200	2.30s	Request Response
5	15:53:29	getCartLineNumbers	ApexRemote	200	0.65s	Request Response
6	15:53:29	getConfigData	ApexRemote	200	1.23s	Request Response
7	15:53:29	getConfigData	ApexRemote	200	2.10s	Request Response
8	15:53:31	getReferenceObjects	ApexRemote	200	2.35s	Request Response
9	15:53:32	getLineItemFieldsMetadata	ApexRemote	200	0.62s	Request Response
10	15:53:32	getSObjectPermissions	ApexRemote	200	0.87s	Request Response
11	15:53:32	getAICartViews	ApexRemote	200	0.56s	Request Response
12	15:53:32	getCart	ApexRemote	200	0.88s	Request Response
13	15:53:32	getConfigData	ApexRemote	200	0.80s	Request Response
14	15:53:32	getSObjectPermissions	ApexRemote	200	0.43s	Request Response
15	15:53:33	getObjectForSummary	ApexRemote	200	0.51s	Request Response
16	15:53:33	getReferenceObjects	ApexRemote	200	1.32s	Request Response
17	15:53:33	getCartLineItems	ApexRemote	200	3.25s	Request Response
18	15:53:36	getProductDetails	ApexRemote	200	1.46s	Request Response
19	15:53:37	getUsagePriceTiers	ApexRemote	200	0.95s	Request Response
20	15:53:37	getReferenceObjects	ApexRemote	200	1.38s	Request Response

#	Time	EndPoint	Action	Status	View
1	15:53:33	/pricing/admin/flags/enable-formulafield-evaluation	GET	200	View
2	15:53:33	pricing/carts/a11D4000000dIFMAA	GET	200	View
3	15:53:33	TaskId: d56eDec7-b26a-4268-abbe-bd968d08a3fc	—	200	View
4	15:53:33	pricing/carts/a11D4000000dIFMAA	PUT	200	View

Right-Side Panel: Call Details

When you select a network call from the main list, a right-side panel will appear, displaying detailed information about that call. This panel is crucial for inspecting the request and response data.

Key functionalities in this panel include:

- **Resend the call:** This button allows you to "Resend the call," which can be useful for re-testing requests or debugging.
- **Copy the JSON:** This button enables you to "Copy the JSON" content of the selected call, making it easy to extract and analyze the data.

The screenshot displays the HAR Extractor application. On the left, a table lists network calls from 'ApexRemote & CongaCloud'. The selected call (index 6) is 'getConfigurationData'. On the right, the 'Raw JSON' view shows the request details. Two red arrows point to the 'Resend' and 'Copy the JSON' buttons in the top right corner of the request view.

#	Time	Method	Type	Status	Duration	Actions
1	15:53:27	initCart	ApexRemote	200	2.18s	Request Response
2	15:53:27	getDefaultLineItemRollup	ApexRemote	200	0.55s	Request Response
3	15:53:27	getAttributeFields	ApexRemote	200	3.49s	Request Response
4	15:53:29	getCart	ApexRemote	200	2.30s	Request Response
5	15:53:29	getCartLineNumbers	ApexRemote	200	0.65s	Request Response
6	15:53:29	getConfigurationData	ApexRemote	200	1.23s	Request Response
7	15:53:29	getConfigurationData	ApexRemote	200	2.10s	Request Response
8	15:53:31	getReferenceObjects	ApexRemote	200	2.35s	Request Response
9	15:53:32	getLineItemFieldsMetadata	ApexRemote	200	0.62s	Request Response
10	15:53:32	getJSONObjectPermissions	ApexRemote	200	0.87s	Request Response
11	15:53:32	getAllCartViews	ApexRemote	200	0.56s	Request Response
12	15:53:32	getCart	ApexRemote	200	0.88s	Request Response
13	15:53:32	getConfigurationData	ApexRemote	200	0.80s	Request Response
14	15:53:32	getJSONObjectPermissions	ApexRemote	200	0.43s	Request Response
15	15:53:33	getJSONObjectForSummary	ApexRemote	200	0.51s	Request Response
16	15:53:33	getReferenceObjects	ApexRemote	200	1.32s	Request Response
17	15:53:33	getCartLineItems	ApexRemote	200	3.25s	Request Response
18	15:53:36	getProductDetails	ApexRemote	200	1.46s	Request Response
19	15:53:37	getUsagePriceTiers	ApexRemote	200	0.95s	Request Response
20	15:53:37	getReferenceObjects	ApexRemote	200	1.39s	Request Response

```

{
  "action": "Apptus_Config.RemotePCController",
  "method": "getConfigurationData",
  "data": {
    "id": (...)
  },
  "type": "rpc",
  "tid": 7,
  "ctx": {
    "csrf": "vexpP5uakfSTUw0S5PhqNV1F4TUrveU18b3UQz0RT1R0YSwU2gkU3w0VwTEVEeId9Mj5TadJU1Y3M8N0U1Ja5I",
    "vid": "8663C80000042ag",
    "as": "Apptus_Config2",
    "ver": 50,
    "authorization": "eyJub25jZ25lMTM4MUR4MGR2OUx4a19ja3l0Zjdhbm1PMX1lRkF6aERenc1B2NE5lNEcxT2lrcTJhMQ01Lj0xOjE3KV"
  }
}

```

Setting Button Modal: Adding New URL Patterns

The "Setting" button (gear icon) in the Network Calls tab opens a modal that allows you to configure URL patterns for tracking network calls. This is particularly useful for focusing on specific types of calls or for organizing your analysis.

Within this modal, you can:

- **Add New Pattern:** Define new URL patterns to track.
- **Name:** Assign a descriptive name to your pattern.
- **URL Pattern:** Specify the actual URL pattern to match.
- **Type:** Select the type of pattern (e.g., Apex (JSON method), HTTP (REST API)).
- **Enabled:** Toggle whether the pattern is active or not.
- **Description (optional):** Add a brief description for clarity.
- **Remove:** Delete an existing pattern.

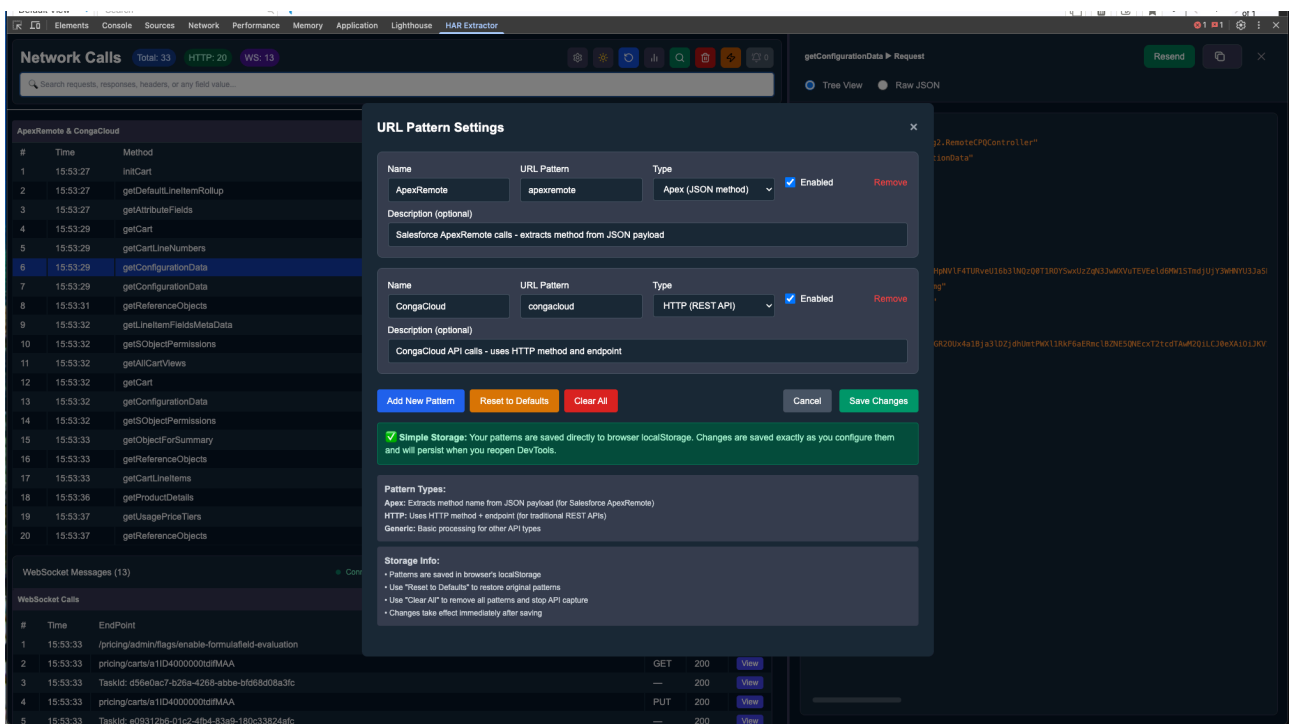
There are also options to "Reset to Defaults" and "Clear All" patterns, as well as "Save Changes" or "Cancel."

Pattern Types:

- **Apex:** Extracts method name from JSON payload (for Salesforce ApexRemote).
- **HTTP:** Uses HTTP method and endpoint (for traditional REST APIs).
- **Generic:** Basic processing for other API types.

Storage Info:

- Patterns are saved in the browser's `localStorage`.
- "Reset to Defaults" restores original patterns.
- "Clear All" removes all patterns and stops API capture.
- Changes take effect immediately after saving.



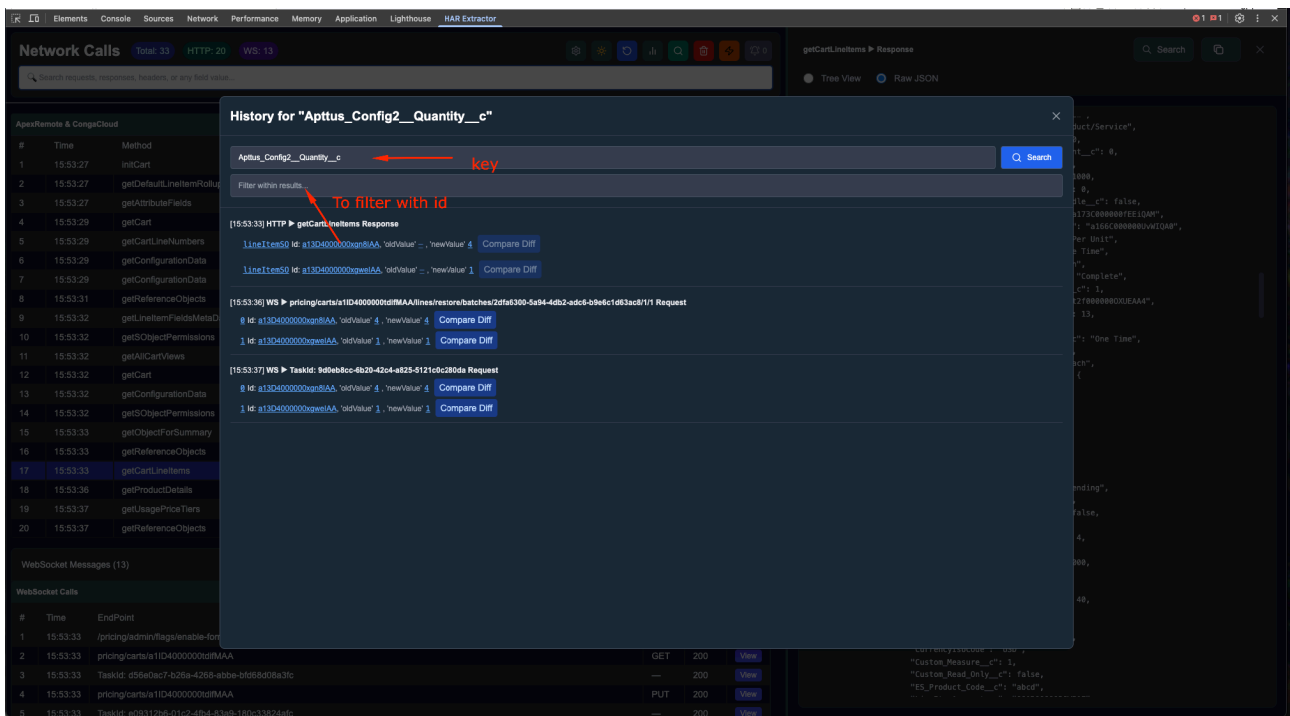
History Modal: Tracking Field History

The "Check history of a particular field" button (magnifying glass with a clock icon) opens the History modal. This modal allows you to track the history of specific data fields within your network calls.

Within the History modal:

- **Key Input:** You can provide a "key" (field name) to search for. Clicking "Search" will display the entire list of historical values for that key.
- **Filter with ID:** Below the key input, there's another input field labeled "To filter with id." If you enter an ID here, the list will be filtered to show only the history relevant to that particular ID.
- **Compare Diff:** For each historical entry, there's a "Compare Diff" button, which likely allows you to see the changes between different versions of the field's value.

This feature is invaluable for debugging and understanding how specific data points evolve over time within your application's network interactions.



HAR Query Search Modal

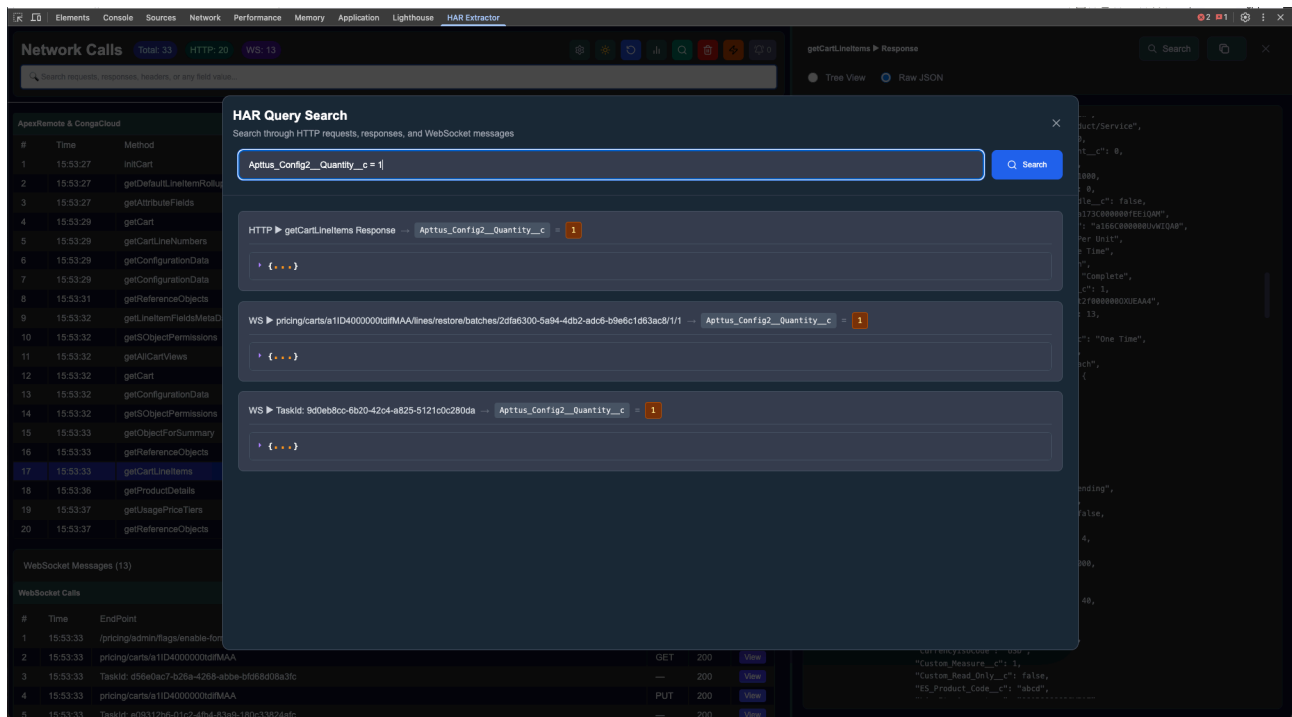
The "Search entire Network calls with conditions" button (magnifying glass icon) opens the HAR Query Search modal. This powerful search tool allows you to search through HTTP requests, responses, and WebSocket messages using specific conditions.

Key features of the HAR Query Search:

- **Dynamic Key Suggestions:** As you type in the search bar, the modal will suggest available keys from your network calls.

- **Conditional Searching:** You can use operators like `=`, `>`, and `<` to find exact objects or filter based on numerical values. For example, `Apttus_Config2__Quantity__c = 1` will find all instances where the `Apttus_Config2__Quantity__c` field has a value of 1.

This advanced search capability helps in quickly pinpointing specific data within a large volume of network traffic.



Add Rule Modal: Tracking Future and Existing Calls

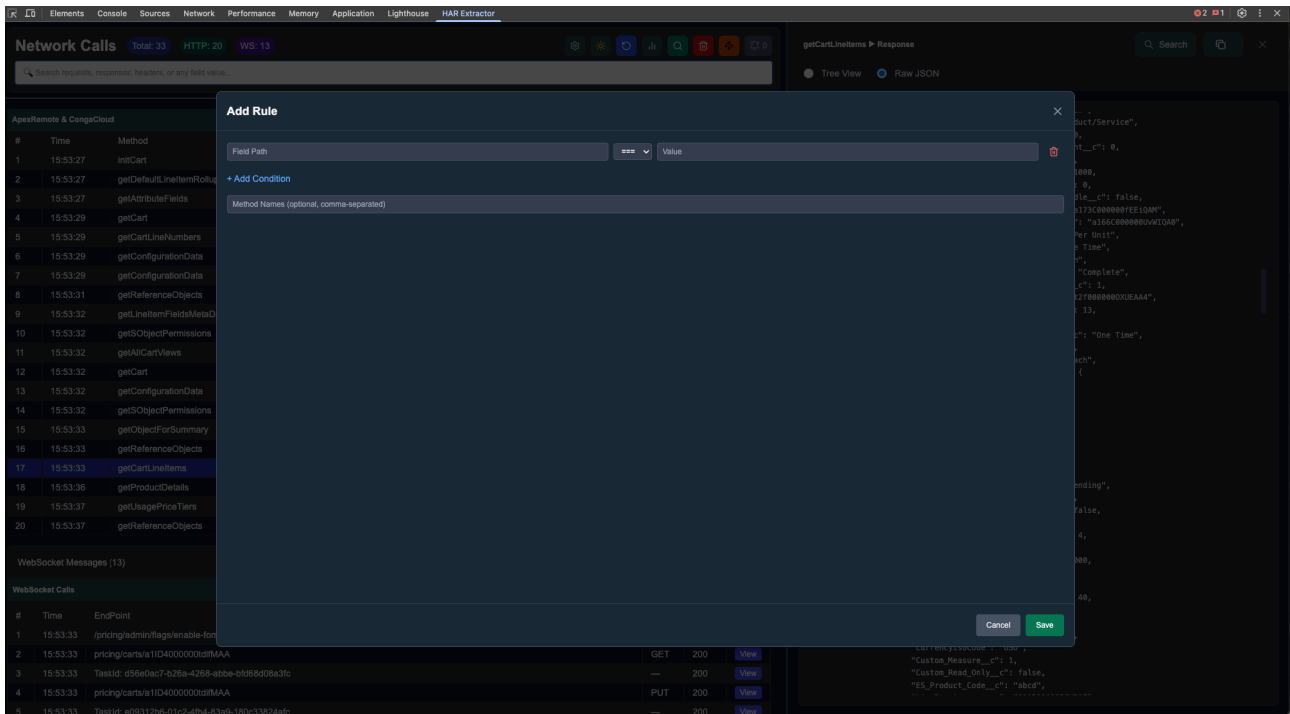
The "Add Rules" button (the icon with three horizontal lines and a plus sign) opens the "Add Rule" modal. This feature allows you to define rules based on key-value pairs to track both future and existing network calls.

Within the "Add Rule" modal, you can:

- **Field Path:** Specify the path to the field you want to track.
- **Value:** Define the value that the field should match.
- **Add Condition:** Add multiple conditions to a single rule.
- **Method Names (optional, comma-separated):** Limit the rule to specific method names.

Once a rule is set, if a network call matches the defined conditions, the extension will trigger a toast notification. This notification will indicate which network call matched the rule and provide a count of how many times it has matched.

This is a powerful feature for monitoring specific events or data changes within your application's network traffic.



Rule Matches and Notifications

When a network call matches a rule you have set, the extension will display a toast notification on the right side of the screen. This notification confirms that a rule has been matched and indicates the network call it corresponds to.

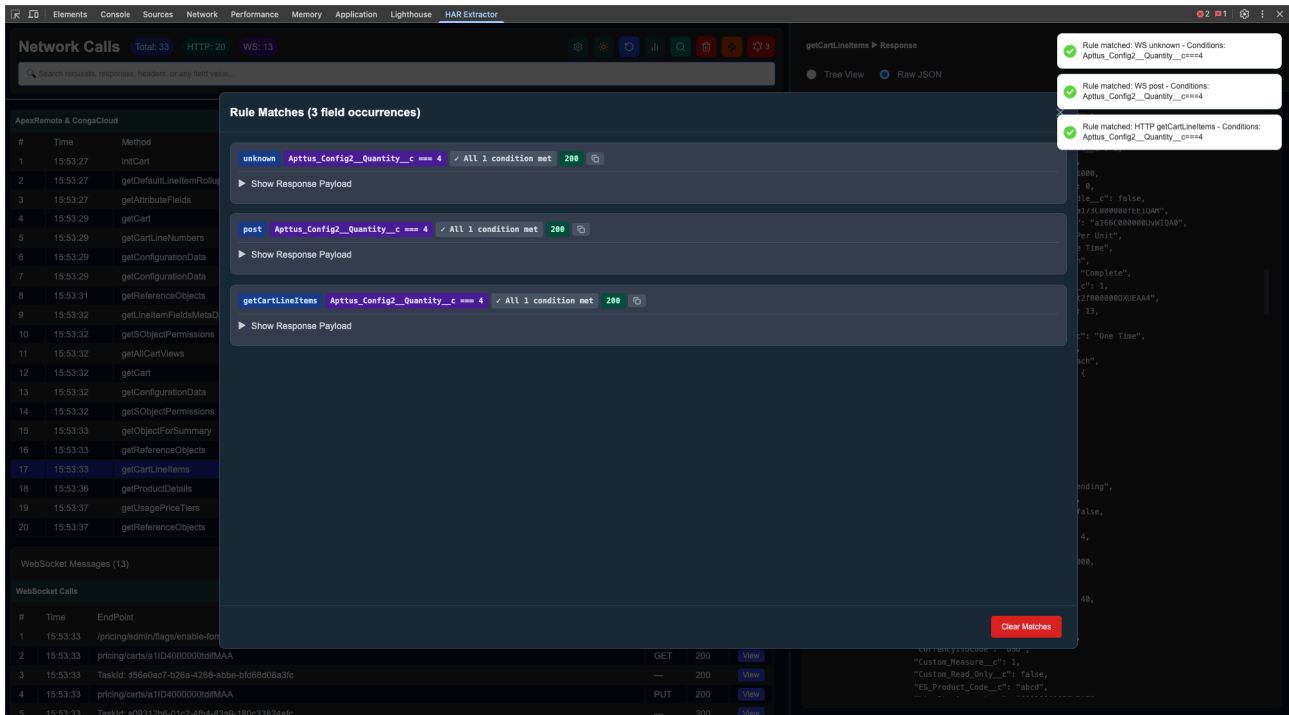
To view a comprehensive list of all matched network calls, click on the bell icon (notification icon) located beside the Rule button. This will open the "Rule Matches" modal.

Within the "Rule Matches" modal:

- **Matched Calls List:** This section displays a list of all network calls that have matched your defined rules. Each entry shows details about the matched call, including the field path and the conditions met.
- **Show Response Payload:** For each matched call, you can click on "Show Response Payload" to view the detailed response data.

- **Clear Matches Button:** At the bottom of the modal, the "Clear Matches" button allows you to clear the entire list of matched network calls.

This feature provides a clear overview of rule-triggered events and helps in quickly reviewing relevant network interactions.



Delete and Reload Network Calls

In the Network Calls tab, you will find buttons to manage the displayed network calls:

- **Delete Button (Trash Can Icon):** Clicking this button will "Delete all the network calls" from the current session. This action removes all HTTP requests and WebSocket messages from the display.
- **Reload Button (Circular Arrow Icon):** This button allows you to reload the network calls. However, it's crucial to understand its behavior:

IMPORTANT NOTE:

The reload button will only retrieve and display HTTP calls. It will NOT reload or retrieve the WebSocket calls list. If you need to see WebSocket calls again after clearing, you may need to restart the debugging session or the browser tab.

The screenshot provided shows a confirmation modal when attempting to clear network logs, which reiterates this important distinction: * HTTP requests can be reloaded from the browser's network tab. * WebSocket messages cannot be reloaded once cleared. * All rule matches and history data will also be lost.

The screenshot displays the Chrome DevTools Network tab with a confirmation modal titled "Clear Network Logs" overlaid. The modal contains the following text:

This action will remove all network logs from your current session.

Important Note:

- HTTP requests can be reloaded from the browser's network tab
- WebSocket messages cannot be retrieved once cleared
- All rule matches and history data will also be lost

The modal has "Cancel" and "Clear All Logs" buttons.

The background shows the Network tab with a list of network calls. The "Network Calls" section shows a list of calls with columns for #, Time, Method, Type, Status, Duration, and Actions. The "WebSocket Messages" section shows a list of messages with columns for #, Time, EndPoint, Action, Status, and View. The "WebSocket Calls" section shows a list of calls with columns for #, Time, EndPoint, Action, Status, and View.

The "Network Calls" table includes the following data:

#	Time	Method	Type	Status	Duration	Actions
1	15:53:27	initCart	XHR	200	2.18s	Request Response
2	15:53:27	getDefaultLineItemRollup	XHR	200	0.55s	Request Response
3	15:53:27	getAttributeFields	XHR	200	3.49s	Request Response
4	15:53:29	getCart	XHR	200	2.30s	Request Response
5	15:53:29	getCartLineNumbers	XHR	200	0.65s	Request Response
6	15:53:29	getConfigurationData	XHR	200	0.55s	Request Response
7	15:53:29	getConfigurationData	XHR	200	0.55s	Request Response
8	15:53:31	getReferenceObjects	XHR	200	0.55s	Request Response
9	15:53:32	getLineItemFieldsMetaData	XHR	200	0.55s	Request Response
10	15:53:32	getObjectContextPermissions	XHR	200	0.55s	Request Response
11	15:53:32	getCartViews	XHR	200	0.55s	Request Response
12	15:53:32	getCart	XHR	200	0.55s	Request Response
13	15:53:32	getConfigurationData	XHR	200	0.55s	Request Response
14	15:53:32	getObjectContextPermissions	XHR	200	0.55s	Request Response
15	15:53:33	getObjectContextPermissions	XHR	200	0.55s	Request Response
16	15:53:33	getReferenceObjects	XHR	200	0.55s	Request Response
17	15:53:33	getCartLineItems	XHR	200	0.55s	Request Response
18	15:53:36	getProductDetails	XHR	200	0.95s	Request Response
19	15:53:37	getUsagePriceTiers	XHR	200	1.30s	Request Response
20	15:53:37	getReferenceObjects	XHR	200	1.30s	Request Response

The "WebSocket Messages" section shows a list of messages with columns for #, Time, EndPoint, Action, Status, and View. The "WebSocket Calls" section shows a list of calls with columns for #, Time, EndPoint, Action, Status, and View.